

SEMANA 1

O QUE FOI FEITO NA SEMANA 1

Durante a primeira semana, construímos toda a estrutura base do dashboard de monitoramento. O objetivo era criar a "carcaça" do sistema - a parte visual e organizacional - sem ainda conectar com dados reais.

1. CRIAÇÃO DO PROJETO

Criamos um projeto React usando a ferramenta Vite, que é mais rápida que o create-react-app tradicional. O projeto foi configurado para usar React com JavaScript.

O projeto está rodando na porta 5173 e pode ser acessado pelo navegador através de <http://localhost:5173>

2. CONFIGURAÇÃO DO TAILWIND CSS

Instalamos e configuramos o Tailwind CSS, que é uma biblioteca de estilos que permite escrever CSS diretamente nos componentes React usando classes prontas como "bg-blue-500" para fundo azul ou "text-white" para texto branco.

A configuração foi feita nos arquivos:

- tailwind.config.js - define onde o Tailwind deve procurar as classes
 - index.css - importa o Tailwind para o projeto
-

3. ESTRUTURA DE PASTAS CRIADA

Organizamos o código em duas grandes áreas:

Pasta **SHARED** (compartilhado)

Guarda tudo que é usado em várias partes do sistema:

- components - componentes reutilizáveis como Card e Layout
- services - serviços como a API de dados falsos (mock)
- types - definições de tipos TypeScript
- utils - funções auxiliares
- hooks - hooks customizados

Pasta FEATURES (funcionalidades)

Guarda o código específico de cada parte do sistema:

- metrics - tudo sobre métricas (CPU, memória)
 - logs - tudo sobre logs do sistema
 - alerts - tudo sobre alertas de segurança
 - hosts - tudo sobre os computadores monitorados
 - dashboard - página principal
-

4. TIPOS DEFINIDOS

Criamos as definições de tipos que descrevem como os dados devem ser:

Host (computador monitorado):

- id: identificador único
- name: nome do computador
- status: se está online ou offline
- ip: endereço de rede

Metric (métrica de desempenho):

- id: identificador único
- hostId: qual computador gerou a métrica
- cpu_percent: porcentagem de uso do processador
- memory_percent: porcentagem de uso da memória
- timestamp: data e hora da coleta

LogEntry (registro de log):

- id: identificador único
- timestamp: data e hora do evento
- type: tipo (autenticação, sistema, ataque)
- message: mensagem descritiva
- source_ip: endereço IP de origem

Alert (alerta de segurança):

- id: identificador único
- severity: gravidade (alta, média, baixa)
- type: tipo do alerta

- source_ip: endereço IP suspeito
 - timestamp: data e hora
 - resolved: se já foi resolvido
-

5. SERVIÇO MOCK CRIADO

Criamos um serviço de dados falsos (mock) para poder desenvolver sem precisar do backend real. Este serviço:

- tem uma lista de 3 computadores (Servidor Principal, Servidor Web, Banco de Dados)
- gera métricas aleatórias de CPU (entre 10% e 90%) e memória (entre 20% e 90%)
- tem logs de exemplo (falha de login, sistema iniciado, ataque detectado)
- simula um atraso de rede para parecer real

Isso permite testar todo o frontend mesmo com o backend ainda em desenvolvimento.

6. COMPONENTES CRIADOS

Componente Card

É um card visual que mostra uma métrica. Ele recebe como informações:

- título (ex: "CPU")
- valor (ex: 45)
- unidade (ex: "%")
- ícone (ex: "⚡")
- cor (azul, verde, vermelho, amarelo)

O card tem um layout bonito com sombra, bordas arredondadas e um círculo colorido com o ícone.

Componente Layout

É o esqueleto da página. Ele contém:

- um cabeçalho fixo no topo com o título "Dashboard de Monitoramento"
- uma área principal onde o conteúdo é inserido
- fundo cinza claro para toda a página

Qualquer página criada usa este Layout para ter consistência visual.

Componente LogsPlaceholder

É um espaço reservado para a tabela de logs que será criada na Semana 3. Por enquanto mostra apenas uma mensagem "Em breve: tabela de logs".

7. DASHBOARD PRINCIPAL

O componente principal App.jsx é onde tudo se junta. Ele faz:

Gerencia estados:

- qual host está selecionado
- as métricas do host selecionado
- se está carregando dados ou não

Carrega dados automaticamente:

Quando a página abre, carrega os hosts disponíveis

Quando o usuário muda de host, carrega as métricas do novo host

Mostra na tela:

- um seletor dropdown para escolher o host
- enquanto carrega, mostra um spinner girando
- depois, mostra 3 cards com CPU, Memória e Status
- abaixo, mostra o placeholder de logs

Tem responsividade:

Em celular: os cards ficam empilhados (1 coluna)

Em tablet: os cards ficam em 2 colunas

Em computador: os cards ficam em 3 colunas lado a lado

8. ESTILOS APLICADOS

Usamos as classes do Tailwind para estilizar tudo:

Cores:

- bg-gray-100 = fundo cinza claro
- bg-white = fundo branco
- bg-blue-100 = fundo azul claro para ícone da CPU
- bg-green-100 = fundo verde claro para ícone da memória
- text-gray-800 = texto cinza escuro

Espaçamento:

- p-4 = padding de 16px em todos os lados
- p-6 = padding de 24px em todos os lados
- m-4 = margem de 16px
- gap-6 = espaçamento de 24px entre itens do grid

Bordas e sombras:

- rounded-lg = borda arredondada
- shadow-md = sombra média
- border = borda simples

Tipografia:

- text-sm = texto pequeno
- text-2xl = texto muito grande
- font-bold = texto em negrito

Layout:

- flex = layout flexível
- grid = layout em grade
- justify-between = espaço entre elementos
- items-center = alinhamento vertical centralizado

Responsividade:

- md:grid-cols-2 = em tablet, 2 colunas
- lg:grid-cols-3 = em desktop, 3 colunas

9. FUNCIONALIDADES PRONTAS

Até o final da Semana 1, o sistema já consegue:

- Mostrar uma interface bonita e organizada
- Listar os computadores disponíveis
- Permitir que o usuário escolha qual computador quer monitorar
- Mostrar os dados de CPU e memória do computador escolhido
- Mostrar se o computador está online ou offline
- Atualizar os dados quando o usuário muda de computador

- Indicar visualmente quando está carregando
 - Funcionar tanto no computador quanto no celular
-

10. TECNOLOGIAS USADAS

React: biblioteca para construir interfaces

Vite: ferramenta para criar e rodar o projeto rapidamente

Tailwind CSS: biblioteca de estilos

JavaScript: linguagem de programação

TypeScript: JavaScript com tipos (usado apenas para definir a forma dos dados)

11. COMANDOS IMPORTANTES

Para rodar o projeto:

`npm run dev`

Para instalar dependências:

`npm install`

Para parar o servidor:

`CTRL + C`

12. O QUE NÃO FOI FEITO NA SEMANA 1

- Conexão com backend real (ainda estamos usando dados falsos)
 - Gráficos (vai ser na Semana 2)
 - Tabela de logs (vai ser na Semana 3)
 - Alertas reais (vai ser na Semana 4)
 - Atualização automática de dados (vai ser na Semana 4)
-

13. PROBLEMAS E SOLUÇÕES

Tailwind não aplicava as classes:

Solução: Verificar se o `index.css` tem a linha `@import "tailwindcss"`

Os cards não atualizavam ao mudar de host:

Solução: Adicionar o host selecionado como dependência do `useEffect`

Layout quebrava no celular:

Solução: Usar classes responsivas como md:grid-cols-2

14. RESUMO DOS ARQUIVOS CRIADOS

Arquivos de tipos (1):

- src/shared/types/index.ts

Arquivos de serviços (1):

- src/shared/services/mockApi.js

Arquivos de componentes compartilhados (2):

- src/shared/components/Card.jsx
- src/shared/components/Layout.jsx

Arquivos de features (1):

- src/features/logs/components/LogsPlaceholder.jsx

Arquivos principais atualizados (2):

- src/App.jsx
 - src/index.css
-

15. CONCLUSÃO

Ao final da Semana 1, temos um dashboard funcional com:

- Interface bonita e responsiva
- Estrutura organizada para crescer
- Dados falsos que permitem testar tudo
- Base pronta para adicionar gráficos, logs e alertas